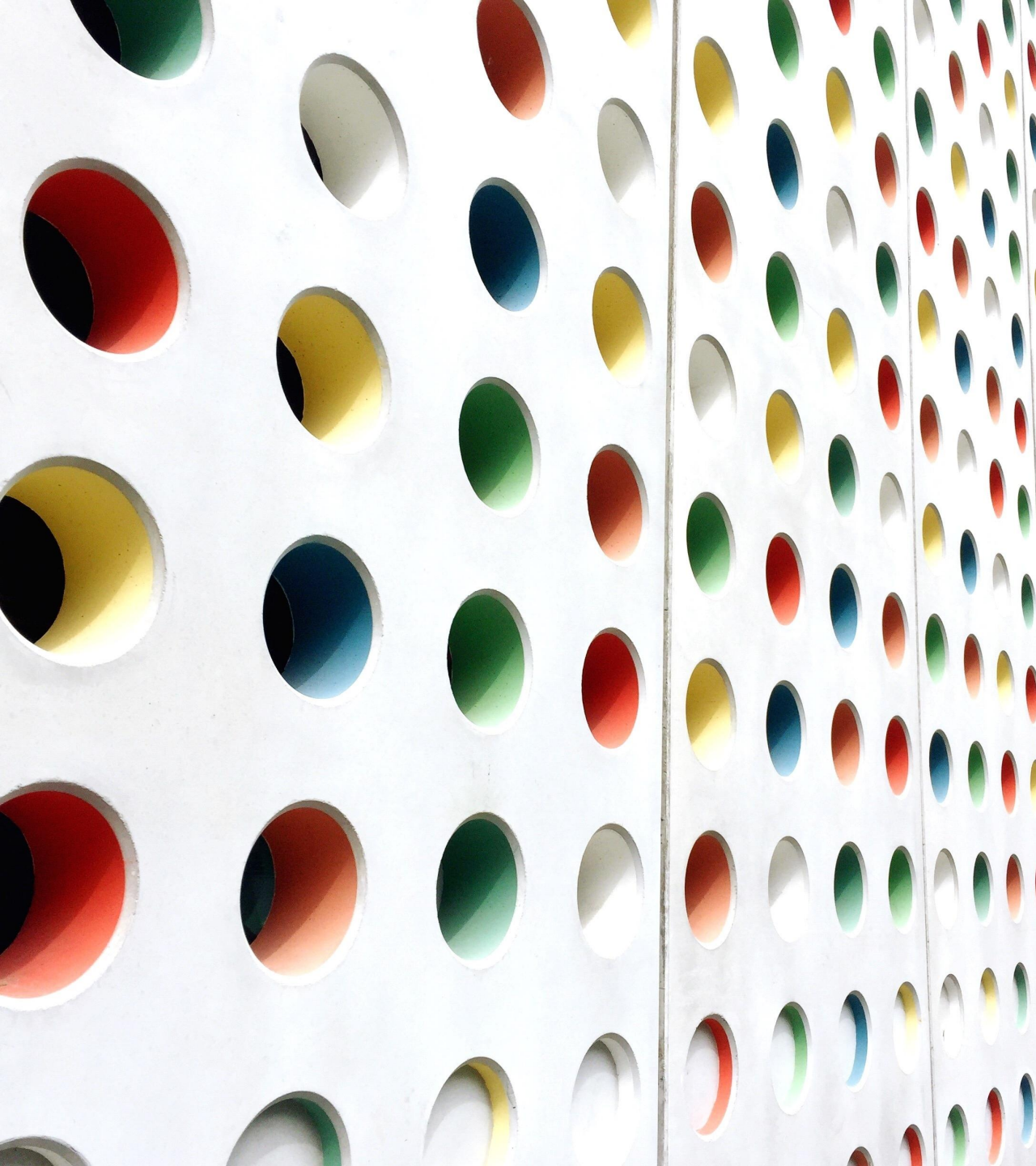




OWL

BY

SANA RIZWAN

What's inside an ontology?

Concepts: Classes + class-hierarchy

- instances

Properties: often also called “Roles” or “Slots”

- labeled instance-value-pairs

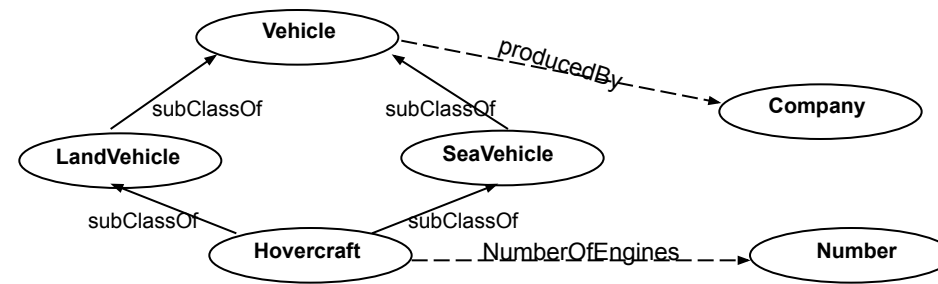
Axioms/Relations:

- relations between classes (disjoint, covers)
- inheritance (multiple? single?)
- restrictions on slots (type, cardinality)
- Characteristics of slots (symm., trans., ...)

reasoning tasks:

- **Classification**: Which classes does an instance belong to?
- **Subsumption**: Does a class subsume another one?
- **Consistency checking**: Is there a contradiction in my axioms/instances?

RDF/RDFS



Recall from last lecture: RDF + RDFS:

RDF: triples for making assertions about resources

RDFS extends RDF with “schema vocabulary”, e.g.:

- Class, Property
- type, subClassOf, subPropertyOf
- range, domain

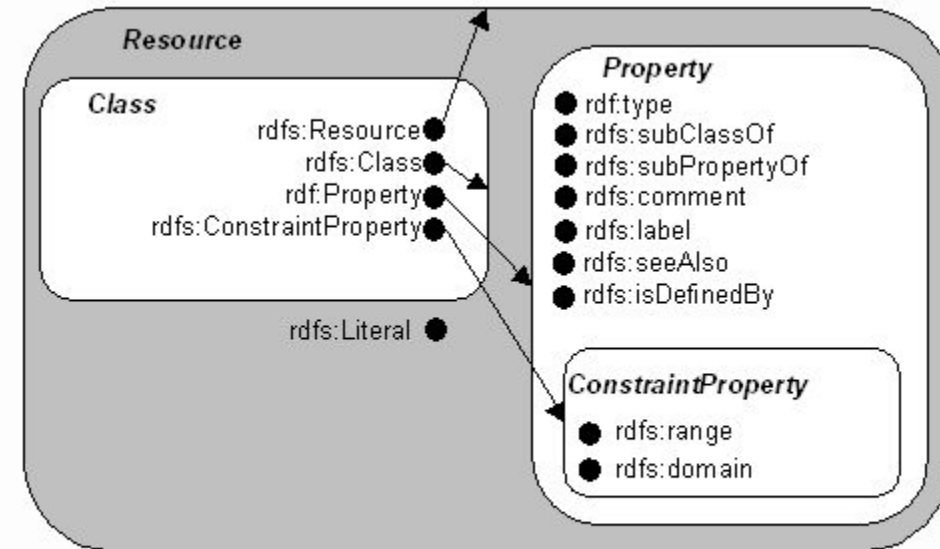
□ representing **simple assertions, taxonomy + typing**

RDF Schema – Strange Things

rdfs:Resource is the superclass of everything, but itself is an instance of its subclass **rdfs:Class**

rdfs:Class itself can be an instance of itself

rdf:Property is an instance of **rdfs:Class** (all RDF properties can be **rdfs:Class**)



Limits of RDF and RDFS

Local scope of properties

- No **localised range and domain** constraints:
- We cannot declare range restrictions that apply to some classes only
- Can't say that the range of hasChild is person when applied to persons and elephant when applied to elephants

Disjointness of classes

- In RDF, we cannot define classes are disjoint, we can only state the subclass relationship
 - E.g., male and female are disjoint

□ OWL (the **W**eb **O**ntology **L**anguage)

Boolean combinations of classes

- RDF does not support boolean combination of classes
 - E.g. cannot describe new classes in terms of combinations (e.g., **union** and **intersection**) of other classes

Cardinality restriction

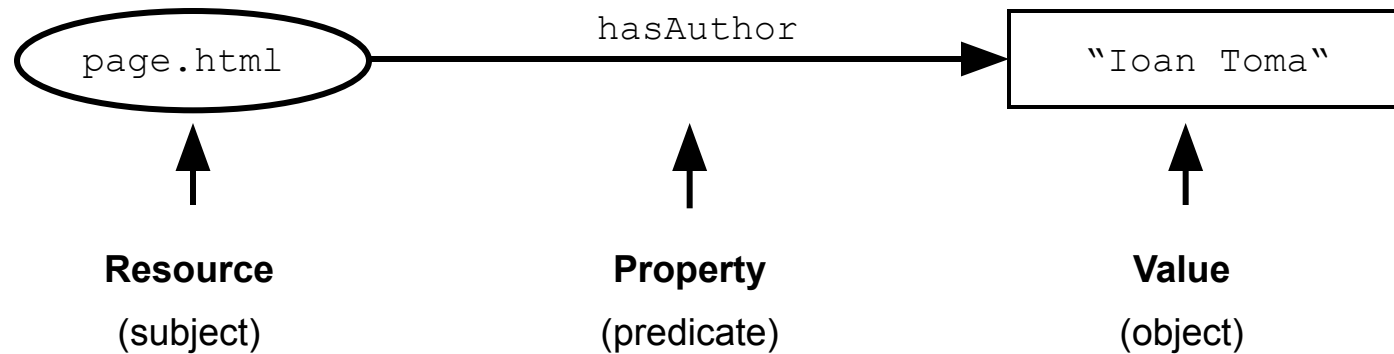
- RDF does not support to place restrictions on how many distinct values a property may or must take
 - E.g. One person can only have no more than 2 parents; one course should have at least one lecturer

Special characteristics of properties (RDF does not support):

- specifying that a given property (such as ex:hasAncestor) is **transitive**, e.g., that if A ex:hasAncestor B, and B ex:hasAncestor C, then A ex:hasAncestor C.
- specifying that a given property is a unique identifier (or **key**) for instances of a particular class.
- Inverse (e.g., eat, eat-by)
- specifying that two different classes (having different URIs) actually represent the same objects (**class identity**)
- specifying that two different instances (having different URIs) actually represent the same individual (**object identity**).

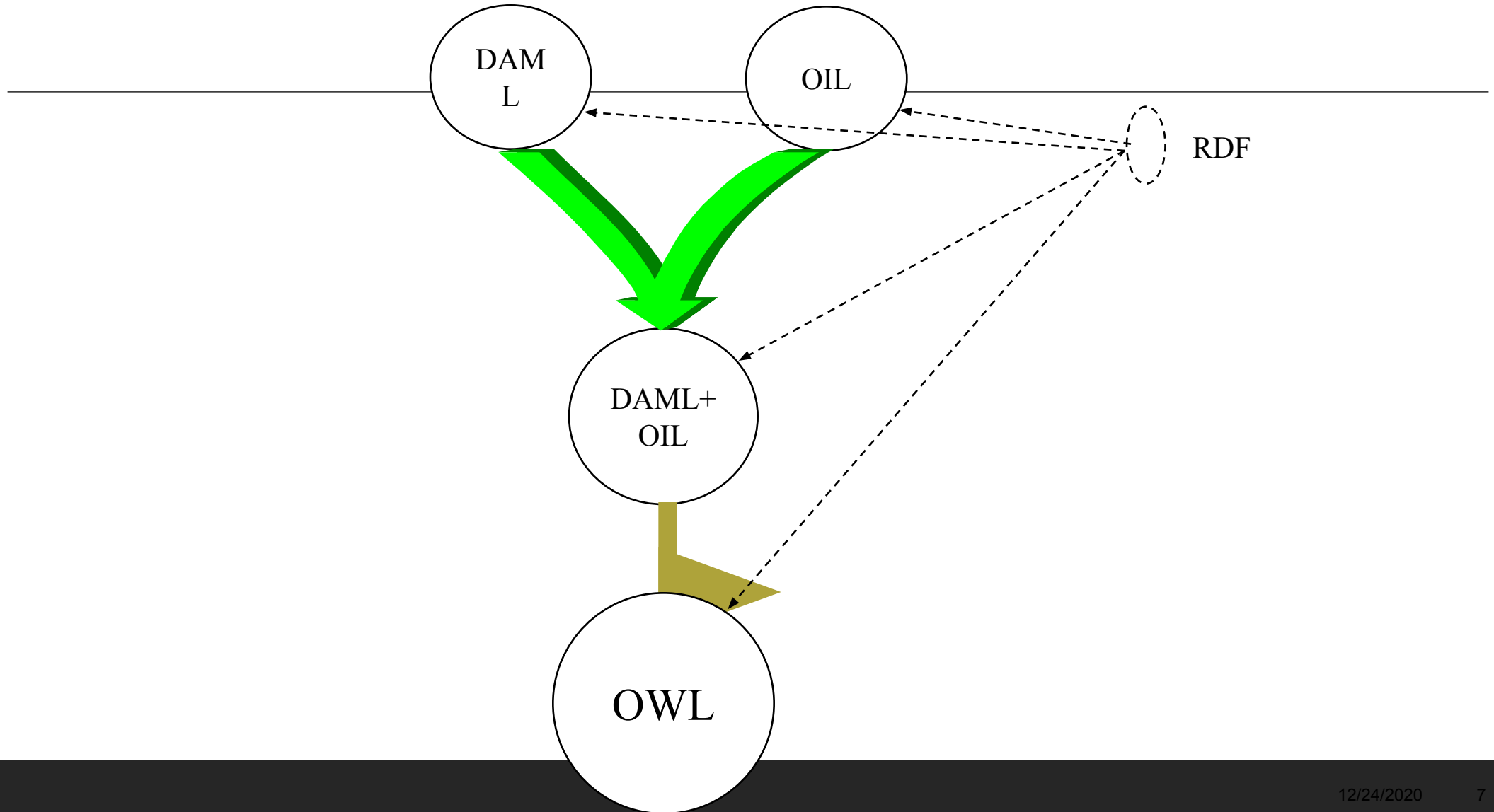
OWL-RDFS Relationship

Both use the same data model:



OWL extends vocabulary and adds axioms to express more complex relations of classes and properties

Origins of OWL



OWL

- Web Ontology Language
- Provides rich set of concepts for defining classes and properties
- Primary keys
- Cardinality restrictions
- Class equivalence, intersections and unions

OWL Language - Overview

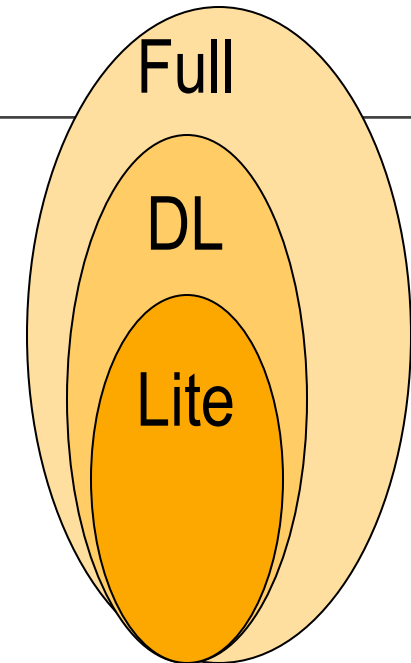
Three species of OWL

- OWL DL stays in **Description Logic** fragment
- OWL Lite is “easier to implement” subset of OWL DL
- OWL Full is union of OWL syntax and RDF

OWL DL benefits from many years of Description Logic research

- Well defined semantics
- Formal properties well understood (complexity, decidability)
- Well-developed reasoning algorithms
- Implemented systems (highly optimised)

OWL Full has all that and all the possibilities of RDF/RDFS which destroy decidability



Syntactic layering
Semantic layering

Three dialects of OWL

OWL Lite

- Sublanguage of OWL DL
- Easier to learn and to implement
- Only provide restricted expressivity.

OWL DL

- Sublanguage of OWL Full which corresponds to Description Logic
- Permits efficient reasoning support
- Lose full compatibility with RDF
- Every legal OWL DL document is a legal RDF document, but not the other way around.

OWL Full

- OWL plus RDF and RDFS
- Fully upward-compatible with RDF (both syntactically and semantically)
- Any legal RDF document is also a legal OWL Full document
- Language is very powerful and is undecidable, impossible to provide complete (or efficient) reasoning support

Description Logic Family

DLs are a family of logic-based KR formalisms

Particular languages mainly characterized by:

- Set of constructors for building complex **concepts** and **roles** from simpler ones
- Set of **axioms** for asserting facts about concepts, roles and individuals

Examples:

- “Female persons”
 - $\text{Person} \sqcap \text{Female}$
- “Non-female persons”
 - $\text{Person} \sqcap \neg \text{Female}$
- “Persons that have a child”
 - $\text{Person} \sqcap \exists \text{hasChild}.\text{Person}$
- “Persons all of whose children are female”
 - $\text{Person} \sqcap \forall \text{hasChild}.\text{Female}$
- “Persons that are employed or self-employed”
 - $\text{Person} \sqcap (\text{Employee} \sqcup \text{SelfEmployed})$
- “Persons the have at most one father”
 - $\text{Person} \sqcap \leq 1.\text{hasFather}$

Necessary and sufficient conditions

Inclusion axioms provide **necessary** conditions:

- $\text{concept} \sqsubseteq \text{definition}$
- Necessary condition: superclass, implies, partial

Equivalence axioms provide **necessary and sufficient** conditions:

- Necessary and sufficient condition: equal, complete

$$\text{concept} \equiv \text{definition} \left\{ \begin{array}{l} \text{concept} \sqsubseteq \text{definition} \\ \text{definition} \sqsubseteq \text{concept} \end{array} \right. \text{ and}$$

OWL Language

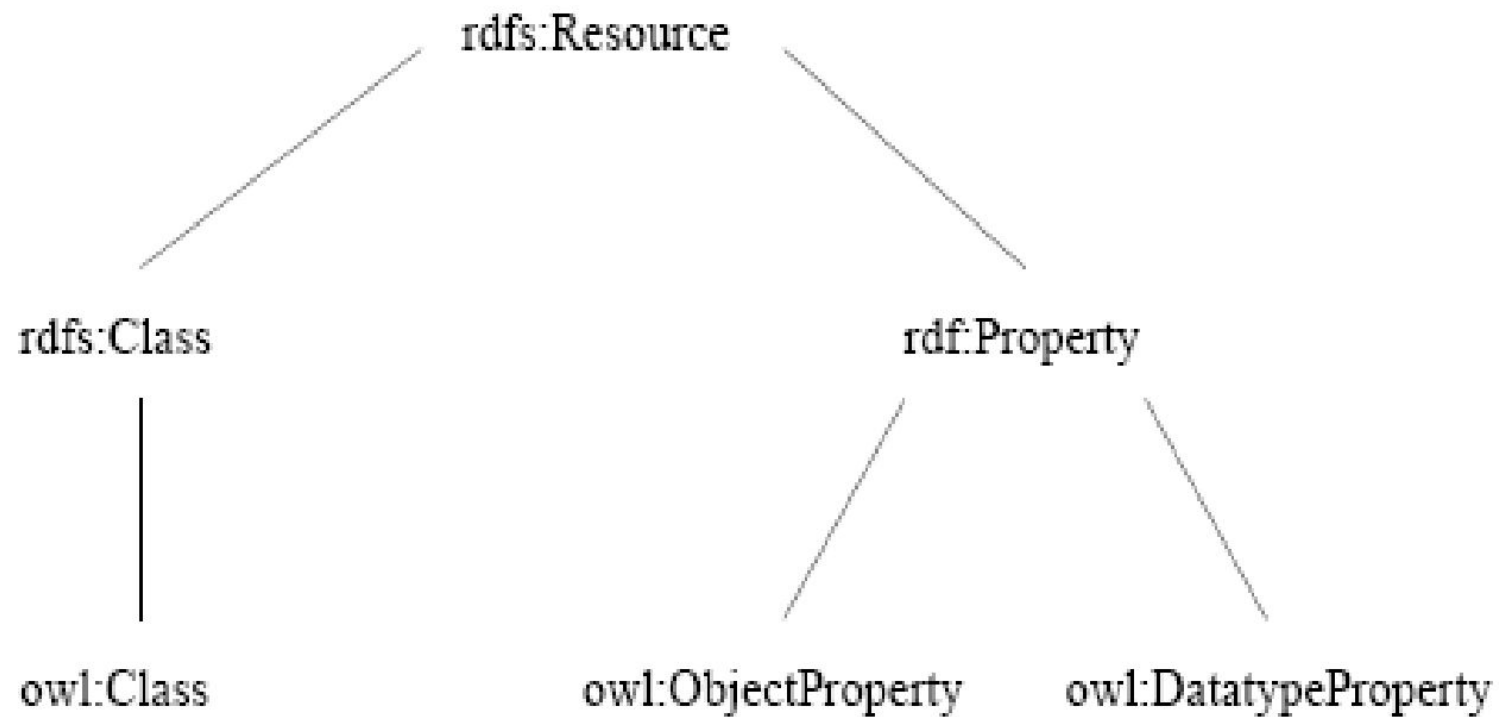
Syntax

- OWL is built on RDF/RDFS and uses RDF/XML-based syntax

Header

```
<rdf:RDF
  xmlns:owl = "http://www.w3.org/2002/07/owl#"
  xmlns:rdf = "http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs = "http://www.w3.org/2000/01/rdf-schema#"
  xmlns:xsd = "http://www.w3.org/2001/XMLSchema#">
```

OWL and RDF/RDFS



OWL Class Elements

owl:Class

```
<owl:Class rdf:ID="associateProfessor">  
  <rdfs:subClassOf rdf:resource="#academicStaffMember"/>  
</owl:Class>
```

owl:disjointWith

```
<owl:Class rdf:about="#associateProfessor">  
  <owl:disjointWith rdf:resource="#professor"/>  
  <owl:disjointWith rdf:resource="#assistantProfessor"/>  
</owl:Class>
```

owl:equivalentClass

```
<owl:Class rdf:ID="faculty">  
  <owl:equivalentClass rdf:resource="#academicStaffMember"/>  
</owl:Class>
```

owl:Thing and owl:Nothing

OWL Property Elements

Object property (relate objects to other objects)

```
<owl:ObjectProperty rdf:ID="teaches">  
  <rdfs:range rdf:resource="#course"/>  
  <rdfs:domain rdf:resource="#academicStaffMember"/>  
  <owl:inverseOf rdf:resource="#isTaughtBy"/>  
</owl:ObjectProperty>
```

Data type property (relate objects to datatype values)

```
<owl:DatatypeProperty rdf:ID="age">  
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema  
    #nonNegativeInteger"/>  
</owl:DatatypeProperty>
```

OWL Instances

Instances of classes are declared as in RDF

Unlike typical database system, OWL does not adopt the unique-names assumption (two instances with different names or IDs does not imply that they are different individuals (they can be the same individuals))

- If a course can only be taught by one teacher, then we can imply that these two instances are the same individual.

```
<course rdf:ID="CIT1111">  
  <isTaughtBy rdf:resource="#949318"/>  
  <isTaughtBy rdf:resource="#949352"/>  
</course>
```

OWL Data types

In fact, not even all of the built-in XML Schema data types can be used in OWL.

The OWL reference document lists the XML Schema data types that can be used, but these includes the most frequently used types, such as string, integer, boolean, time and date.

OWL Versioning

Can only indicating informal versioning information, none of them carry any formal meaning:

- Owl:priorVersion- indicate earlier versions of the current ontology
- Owl:versionInfo - a string giving information about current version
- Owl:backwardCompatibleWith - a reference to another ontology (reuse this ontology by using owl:imports)
- Owl:incompatibleWith- the containing ontology is the later version of the referenced ontology but is not backward-compatible with it.

Describing Classes in OWL

Classes: OWL vs. RDFS

OWL allows greater expressiveness, but

OWL (DL/Lite) puts certain constraints on the use of RDF

- For instance: **a class may not act as an instance** of another (meta)class (the same holds for properties)

□ OWL has got its own Class identifier

RDFS

```
<rdfs:Class rdf:ID="River">  
  <rdfs:subClassOf rdf:resource="#Stream"/>  
</rdfs:Class>
```

OWL

```
<owl:Class rdf:ID="River">  
  <rdfs:subClassOf rdf:resource="#Stream"/>  
</owl:Class>
```

RDF constructs

What can you express in RDF/RDFS?

Not too much...

Employee $\sqsubseteq$ Person

- `rdfs:subClassOf`
... class hierarchy, necessary conditions (also equivalence is expressible because $A \sqsubseteq B$ and $B \sqsubseteq A \sqRightarrow A \equiv B$)

Employee(alice)

- `rdf:type`
... class membership
... OWL provides more expressive constructs to express the DL features!

Union of Classes

-
- Instances of the Union of two Classes are either the instance of one or both classes

Person $\equiv$ Man $\sqcup$ Woman

```
<owl:Class rdf:ID="Person">  
  <owl:unionOf rdf:parseType="Collection">  
    <owl:Class rdf:about="#Woman" />  
    <owl:Class rdf:about="#Man" />  
  </owl:unionOf>  
</owl:Class>
```

Intersection of Classes

-
- Instances of the Intersection of two Classes are simultaneously instances of both class

Man $\equiv$ Person $\sqcap$ Male

```
<owl:Class rdf:ID="Man">  
  <owl:intersectionOf rdf:parseType="Collection">  
    <owl:Class rdf:about="#Person" />  
    <owl:Class rdf:about="#Male" />  
  </owl:intersectionOf>  
</owl:Class>
```

one of: Enumerated Classes

Specify a class via a direct enumeration of its members:

WhineColor $\equiv$ {White, Rose, Red}

```
<owl:Class rdf:ID="WineColor">
  <rdfs:subClassOf rdf:resource="#WineDescriptor"/>
  <owl:oneOf rdf:parseType="Collection">
    <owl:Thing rdf:about="#White"/>
    <owl:Thing rdf:about="#Rose"/>
    <owl:Thing rdf:about="#Red"/>
  </owl:oneOf>
</owl:Class>
```

Necessary vs. necessary and sufficient conditions in OWL...

Necessary condition ($\sqsubseteq$) via
rdfs:subclassOf

Necessary and sufficient ($\equiv$) either by
 $\sqsubseteq + \sqsupseteq$, i.e. 2 subclass definitions or:
owl:equivalentClasses

Property Restrictions

Defining a Class by restricting its possible instances via their property values

OWL distinguishes between the following two:

- Value constraint
 - $(\text{Mother} \equiv \text{Woman} \sqcap \exists \text{hasChild}.\text{Person})$
- Cardinality constraint
 - $(\text{MotherWithManyChildren} \equiv \text{Mother} \sqcap \geq 3 \text{hasChild})$

Property restrictions are local

- remember RDFS allows only global properties

- A Mother is a Woman that has a child (some Person)

Mother $\sqsubseteq$ Woman $\sqcap \exists \text{hasChild}.\text{Person}$

```
<owl:Class rdf:ID="Mother">
  <rdfs:subClassOf rdf:resource="#Woman" />
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#hasChild" />
      <owl:someValuesFrom rdf:resource="#Person" />
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

allValuesFrom

- To express the set of parents that only have female children (daughters) you would write:

ParentsWithOnlyDaughters $\sqsubseteq$ Person $\sqcap \forall \text{hasChild.Woman}$

```
<owl:Class rdf:ID="ParentsWithOnlyDaughters">
  <rdfs:subClassOf rdf:resource="#Person" />
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#hasChild" />
      <owl:allValuesFrom rdf:resource="#Woman" />
    </owl:Restriction>
  </rdfs:subClassOf>
  ...
</owl:Class>
```

hasValue

hasValue allows to define classes based on the existence of *particular* property values, their must be *at least one* matching property value

The set of all children of the woman MARRY would be expressed like the following:

MarysChildren $\sqsubseteq$ Person $\sqcap \exists$ hasParent.{MARRY}

```
<owl:Class rdf:ID="MarysChildren">
  <rdfs:subClassOf rdf:resource="#Person" />
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#hasParent" />
      <owl:hasValue rdf:resource="#MARRY" />
    </owl:Restriction>
  </rdfs:subClassOf>
</rdfs:subClassOf>
...
</owl:Class>
```

cardinality

Definition of cardinality: the number of occurrences, either maximum (maxCardinality) or minimum (minCardinality) or exact (cardinality) based upon the context (class) in which it is used

To define a half-orphan you would write the following:

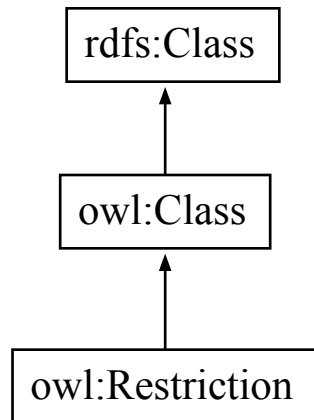
HalfOrphan $\sqsubseteq$ Person $\sqcap$ =1hasParent.Person

```
<owl:Class rdf:ID="HalfOrphan">
  <rdfs:subClassOf rdf:resource="#Person" />
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#hasParent"/>
      <owl:cardinality rdf:datatype="&xsd;NonNegativeInteger">1</owl:cardinality>
    </owl:Restriction>
  </rdfs:subClassOf>
</rdfs:subClassOf>
...
</owl:Class>
```

Classes as Restrictions on Properties

In summary:

define a class using *LOCAL* restrictions on a specific Property



- **Properties:**
 - allValuesFrom: *rdfs:Class (lite/DL owl:Class)*
 - hasValue: *specific Individual*
 - someValuesFrom: *rdfs:Class (lite/DL owl:Class)*
 - cardinality: *xsd:nonNegativeInteger (in lite {0,1})*
 - minCardinality: *xsd:nonNegativeInteger (in lite {0,1})*
 - maxCardinality: *xsd:nonNegativeInteger (in lite {0,1})*

Describing Properties in OWL

Properties OWL vs. RDF

RDF Schema provides a couple of pre-defined properties:

- `rdfs:range` used to indicate the range of values for a property.
- `rdfs:domain` used to associate a property with a class.
- `rdfs:subPropertyOf` used to specialize a property.

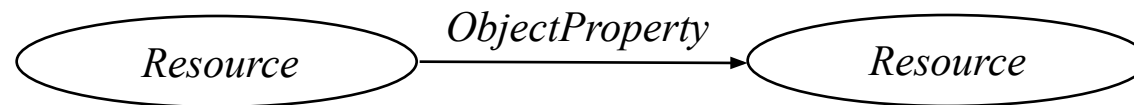
OWL provides additional property classes, which allow reasoning and inferencing, i.e.

- `owl:functionalProperty`
- `owl:transitiveProperty`
- `owl:symmetricProperty`

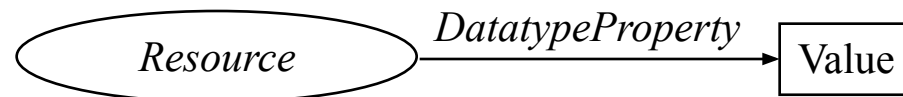
Properties OWL vs. RDF

OWL (DL and Lite) distinguishes between **data type properties** and **object properties** (RDFS does not)

An ObjectProperty relates one **Resource** to another **Resource**:



A DatatypeProperty relates a **Resource** to a **Literal** or an **XML Schema** data type:



Datatype Properties – Example:

for instance, usage of XSD datatypes:

```
<owl:DatatypeProperty rdf:ID="yearValue"> <rdfs:domain  
  rdf:resource="#VintageYear" /> <rdfs:range  
  rdf:resource="&xsd;positiveInteger"/>  
</owl:DatatypeProperty>
```

... *object properties* in OWL allow for some more sophisticated definitions than only domain and range, see following slides...

Transitive Property

Example: If person A is an ancestor of person B and B is an ancestor of C then A is also an ancestor of C.

A=B and B=C, then A=C

$A \in B$ and $B \in C$ then $A \in C$

$\text{ancestor}^+ \subseteq \text{ancestor}$

```
<owl:ObjectProperty rdf:ID="ancesotor">  
  <rdf:type rdf:resource="&owl;TransitiveProperty" />  
  <rdfs:domain rdf:resource="#Person" />  
  <rdfs:range rdf:resource="#Person" />  
</owl:ObjectProperty>
```

Symmetric Property

Example: If Mary is akin to John then John is also akin to Mary

$A=B$ then $B=A$, $A \subseteq B$ then $B \subseteq A$

$\text{akin}^{-} \subseteq \text{akin}$ and $\text{akin} \subseteq \text{akin}^{-}$

```
<owl:ObjectProperty rdf:ID="akin">  
  <rdf:type rdf:resource="&owl;SymmetricProperty" />  
  <rdfs:domain rdf:resource="#Person" />  
  <rdfs:range rdf:resource="#Person" />  
</owl:ObjectProperty>
```

Inverse Property

Example: If Mary **is a child** of John then John **is the Father** of Mary

Teacher **teaches** Course.

Course **isTaughtBy** Teacher. ☐ inverse property

hasChild $\subseteq$ hasParent⁻

```
<owl:ObjectProperty rdf:ID="hasChild">  
  <owl:inverseOf rdf:resource="hasParent" />  
</owl:ObjectProperty>
```

Functional Properties

A functional property states that the value of range for a certain object in the domain is always the same:

Example: A child has always the same Father (biological)

hasFather: A hasFather B, A hasFather C $\square B=C$

A(subject) class is domain and B(object) class is range

Person $\sqsubseteq \leq 1$ hasFather

```
<owl:ObjectProperty rdf:ID="hasFather">  
  <rdf:type rdf:resource="&owl;FunctionalProperty"/>  
</owl:ObjectProperty>
```

Inverse Functional Properties

Defines a property for which two different objects cannot have the same value

E.g., two persons cannot have the same social security number

hasID: A hasID B. C hasID B $\square$ A=C

Person has =1SocialSecurityNumber

```
<owl:ObjectProperty rdf:ID="isTheSocialSecurityNumberfor">  
  <rdf:type rdf:resource="&owl;InverseFunctionalProperty"/>  
</owl:ObjectProperty>
```

Example:

Teacher teaches course.

Course_code hasIDof course.

A is data of teacher and domain of teaches and B is course which is range of teaches.

C is data of course_code and domain of hasIDof and B is course which is range of hasIDof.

Then

Teacher = course_code i.e $A(T121) = C(CSE344)$ (both are domain) $\square$ inverse functional property

Functional vs. inverse functional properties

FunctionalProperty vs InverseFunctionalProperty

	domain	range	example
Functional Property	For a given domain	Range is unique	hasFather: A hasFather B, A hasFather C $\square B=C$
InverseFunctional Property	Domain is unique	For a given range	hasID: A hasID B, C hasID B $\square A=C$

Primary keys

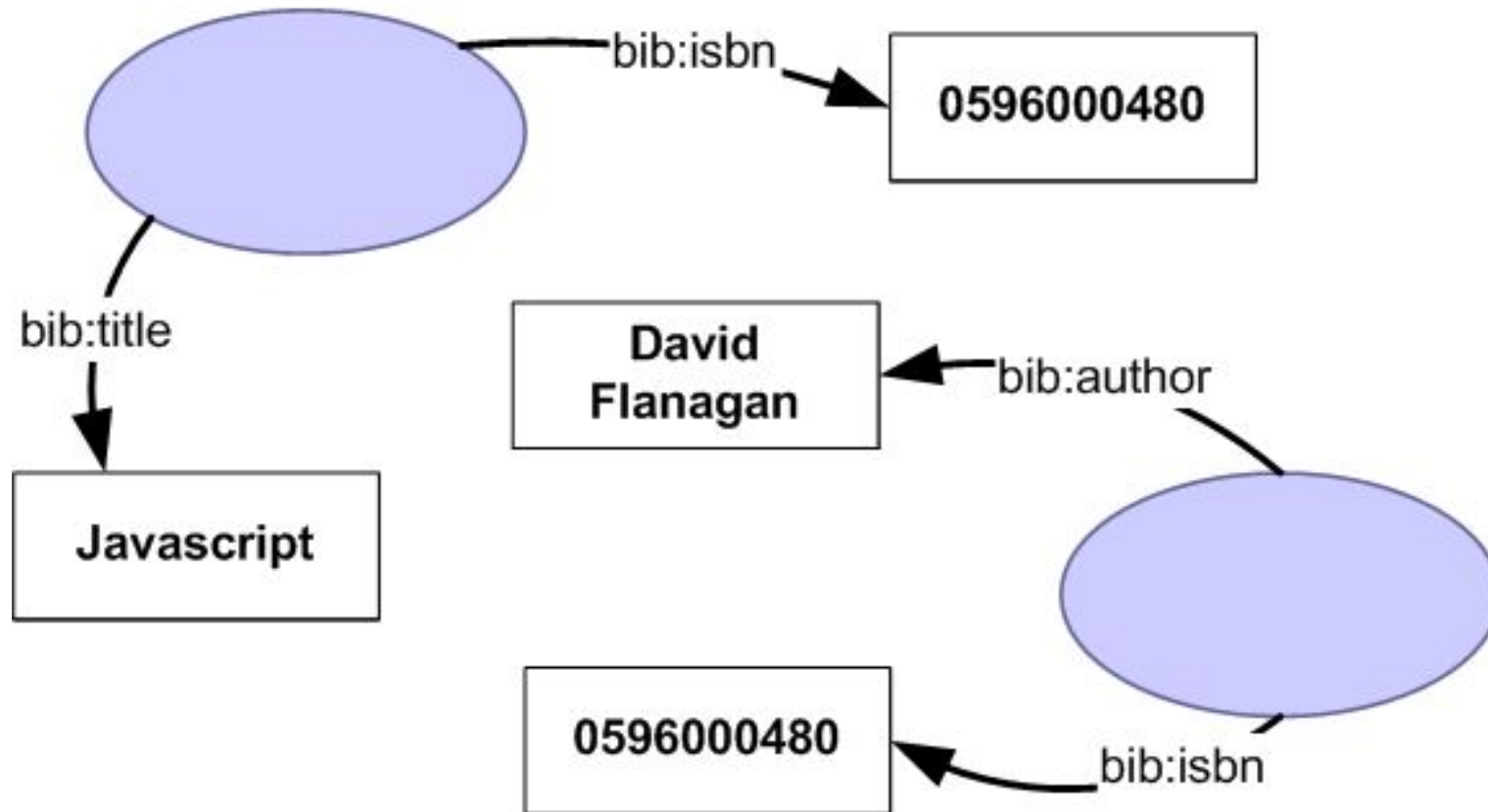
owl:InverseFunctionalProperty (IFP)

If a property is inverse functional then the value of that property identifies the subject

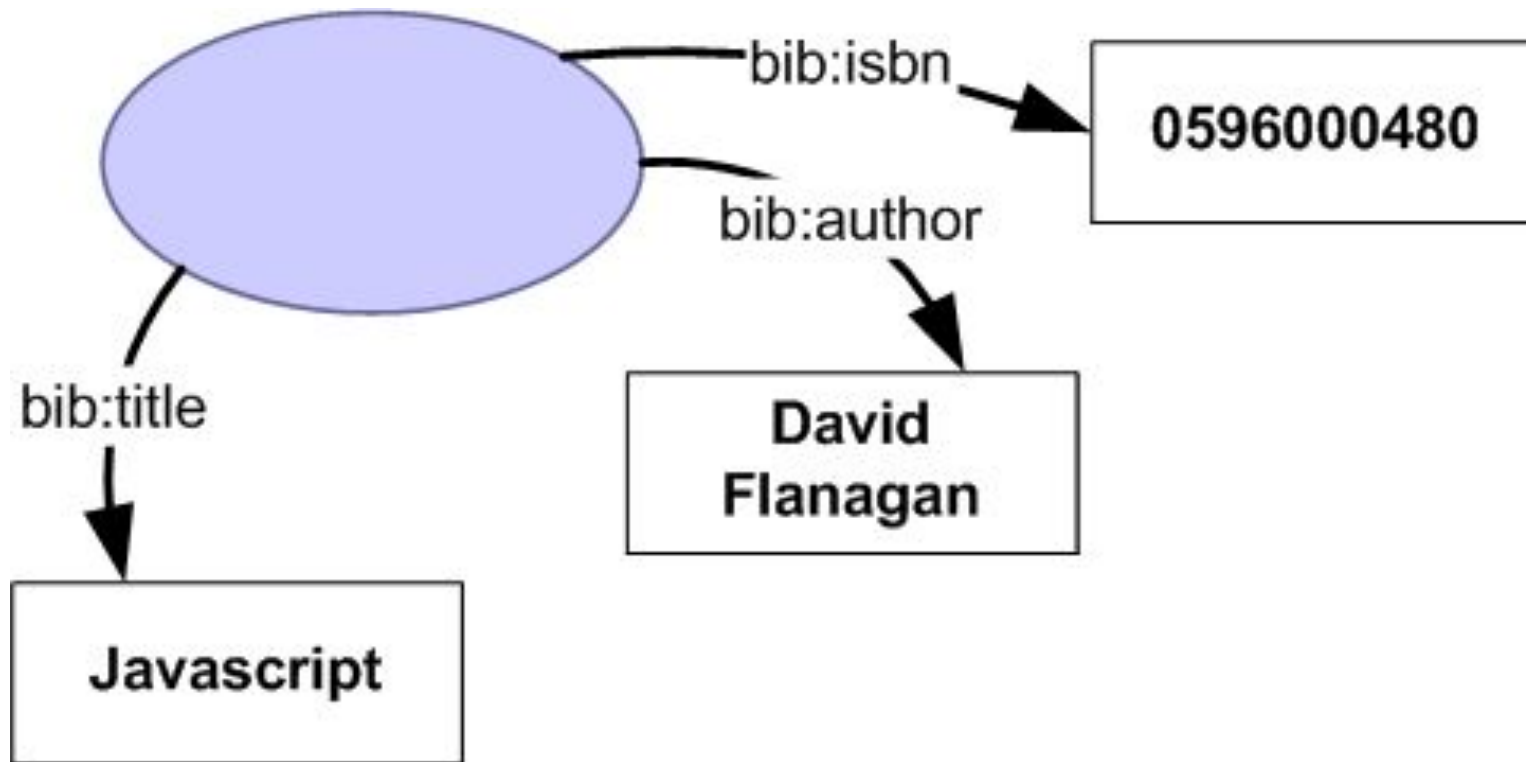
e.g. the isbn relation is inverse functional

- For any ISBN, there is only one book that has that ISBN

IFP example



IFP example



OWL Properties

OWL Properties represent relationships between two objects.

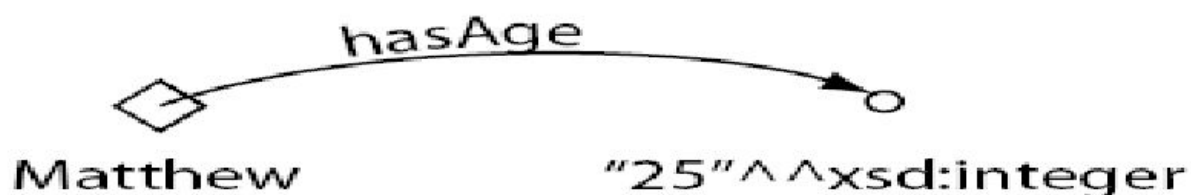
There are two main properties:

- Object properties: link object to object
- datatype properties: link object to XML Schema datatype or `rdf:literal`

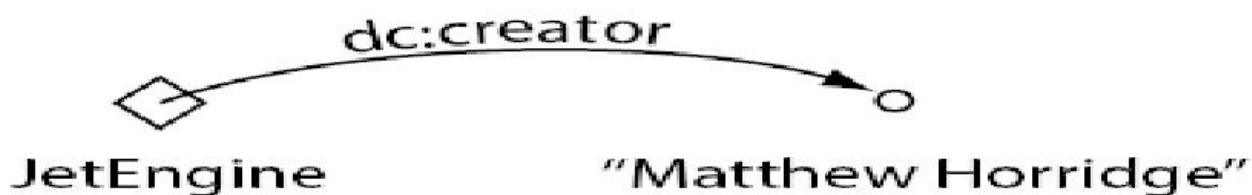
OWL has another property – Annotation properties, to be used to add annotation information to classes, individuals, and properties



An object property linking the individual Matthew to the individual Gemma



A datatype property linking the individual Matthew to the data literal '25', which has a type of an xml:integer.

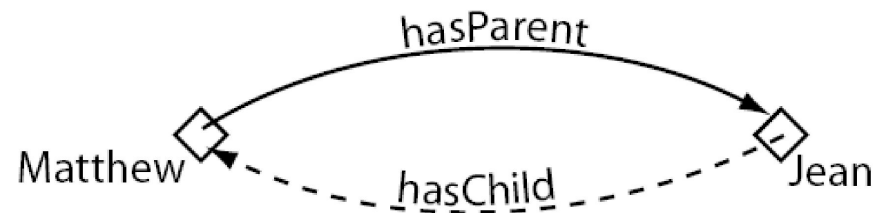


An annotation property, linking the class 'JetEngine' to the data literal (string) "Matthew Horridge".

Inverse Properties

Each object property may have a corresponding inverse property.

If some property links individual a to individual b, then its inverse property will link individual b to individual a.

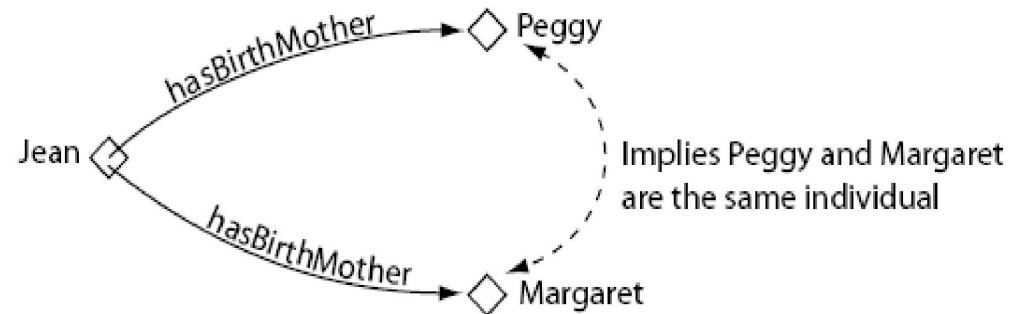


Functional Properties

If a property is functional, for a given individual, there can only be **at most one individual** to be related via this property.

- For a given domain, range must be unique

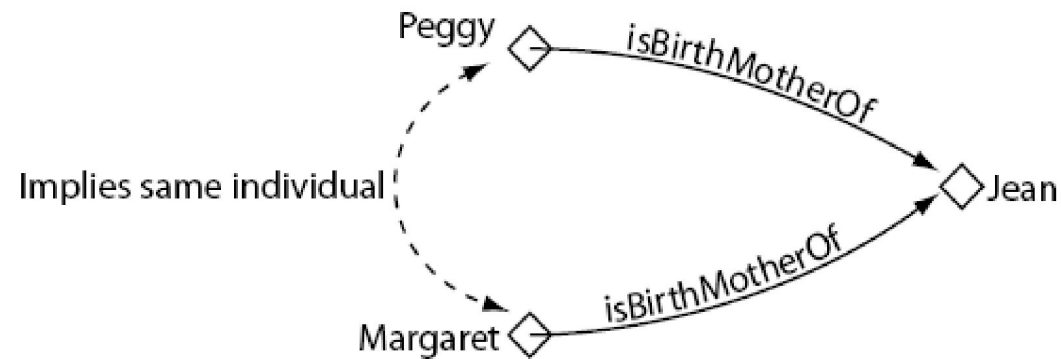
Functional properties are also known as **single valued** properties.



Inverse Functional Properties

If a property is inverse functional, then its inverse property is functional.

- For a given range, domain must be unique.



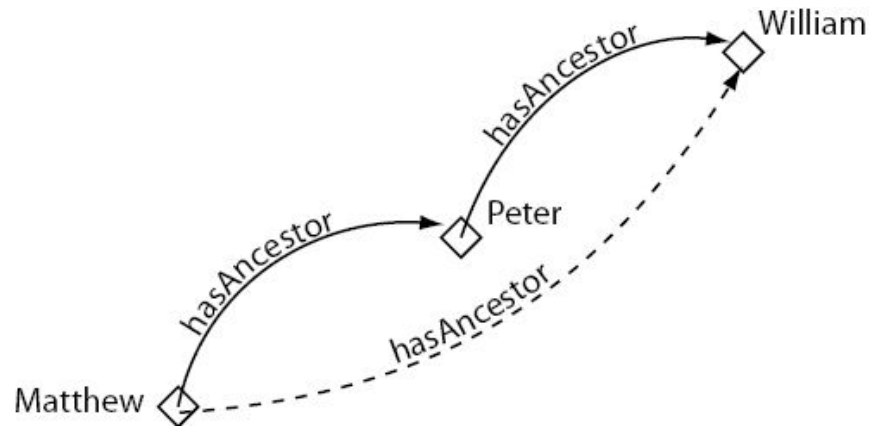
Functional vs. inverse functional properties

FunctionalProperty vs InverseFunctionalProperty

	domain	range	example
Functional Property	For a given domain	Range is unique	hasFather: A hasFather B, A hasFather C $\square B=C$
InverseFunctional Property	Domain is unique	For a given range	hasID: A hasID B, C hasID B $\square A=C$

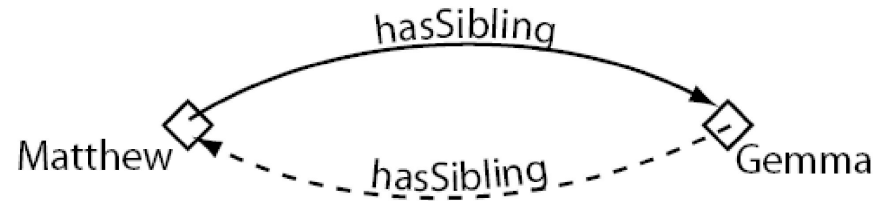
Transitive Properties

If a property is transitive, and the property related individual a to individual b, and also individual b to individual c, then we can infer that individual a is related to individual c via property P.



Symmetric Properties

If a property P is symmetric, and the property relates individual a to individual b , then individual b is also related to individual a via property P .



Use Case – OWL Reasoning

ACKNOWLEDGEMENT FROM TALIS TUTORIAL: TALIS:
[HTTP://RESEARCH.TALIS.COM/2005/RDF-INTRO](http://research.talis.com/2005/RDF-INTRO)

Simple Inferencing

OWL enables simple inferencing

Drawing conclusions based on the restrictions and descriptions

OWL Inferencing 1

First of all a robbery takes place. The robber drops his gun while fleeing. A report is filed by the investigating officers:

```
<RobberyEvent>
  <date>14th June 2005</date>
  <description>Armed robbery at Kwik-e-Mart</description>
  <evidence>
    <Gun>
      <serial>983GTE-H5TF</serial>
    </Gun>
  </evidence>
  <robber>
    <Person /> <!-- an unknown person -->
  </robber>
</RobberyEvent>
```

OWL Inferencing 2

Subsequently a car is pulled over for speeding. The traffic officer files a report electronically while issuing a ticket:

```
<SpeedingOffence>
  <date>26 October 2005</date>
  <description>Car observed driving at speed along
M42</description>
  <speeder>
    <Person>
      <name>John Doe</name>
      <driversLicenseNumber>7431224667</driversLicenseNumber>
    </Person>
  </speeder>
</SpeedingOffence>
```

OWL Inferencing 3

At police HQ, the computer analyses each report as it is filed. The following **OWL description** tells the computer that a driverLicenseNumber is unique to a Person:

Person □ driversLicenseNumber □ 7431224667

Domain(subject) □ Predicate □ Range (object) (URI/Literal)

```
<owl:InverseFunctionalProperty rdf:ID="driversLicenseNumber">
  <rdfs:domain rdf:resource="Person" />
  <rdfs:range rdf:resource="&rdf;Literal" />
</owl:FunctionalProperty>
```

OWL Inferencing 4

The computer uses this information to look up any other records it has about that person and finds a gun license:

```
<GunLicense>
  <registeredGun>
    <Gun>
      <serial>983GTE-H5TF</serial>
    </Gun>
  </registeredGun>
  <holder>
    <Person>
      <name>John Doe</name>
      <driversLicenseNumber>7431224667</driversLicenseNumber>
    </Person>
  </holder>
</GunLicense>
```

OWL Inferencing 5

The next OWL description tells the computer that the registeredGun property is uniquely identifies a GunLicense. i.e. each gun is associated with only a single GunLicense

```
<owl:InverseFunctionalProperty rdf:ID="registeredGun">  
  <rdfs:domain rdf:resource="GunLicense" />  
  <rdfs:range rdf:resource="Gun" />  
</owl:FunctionalProperty>
```

OWL Inferencing 6

The computer now knows that the person stopped for speeding owns a gun. The next description tells the computer that each gun is uniquely identified by its serial

```
<owl:InverseFunctionalProperty rdf:ID="serial">
  <rdfs:domain rdf:resource="Gun" />
  <rdfs:range rdf:resource="&rdf;Literal" />
</owl:FunctionalProperty>
```

OWL Inferencing 7

The computer uses this to determine that the gun on the license is the same gun used in the robbery. This final description, seals the speeder's fate. It tells the computer that each GunLicense applies to **only one gun and one person**, so there is no doubt that the speeder is the person who owns the gun:

```
<owl:Class rdf:ID="GunLicense">
  <owl:intersectionOf rdf:parseType="Collection">
    <owl:Restriction>
      <owl:onProperty rdf:resource="#registeredGun"/>
      <owl:cardinality>1</owl:cardinality>
    </owl:Restriction>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#holder"/>
      <owl:cardinality>1</owl:cardinality>
    </owl:Restriction>
  </owl:intersectionOf>
</owl:Class>
```

OWL Inferencing 8

The computer reports back to the traffic cop who dully arrests the speeder on suspicion of armed robbery



OWL Lite

RDF Schema Features:

- [Class \(Thing, Nothing\)](#)
- [rdfs:subClassOf](#)
- [rdf:Property](#)
- [rdfs:subPropertyOf](#)
- [rdfs:domain](#)
- [rdfs:range](#)
- [Individual](#)

(In)Equality:

- [equivalentClass](#)
- [equivalentProperty](#)
- [sameAs](#)
- [differentFrom](#)
- [AllDifferent](#)
- [distinctMembers](#)

• Property Characteristics:

- [ObjectProperty](#)
- [DatatypeProperty](#)
- [inverseOf](#)
- [TransitiveProperty](#)
- [SymmetricProperty](#)
- [FunctionalProperty](#)
- [InverseFunctionalProperty](#)

• Property Restrictions:

- [Restriction](#)
- [onProperty](#)
- [allValuesFrom](#)
- [someValuesFrom](#)

• Restricted Cardinality:

- [minCardinality](#) (only 0 or 1)
- [maxCardinality](#) (only 0 or 1)
- [cardinality](#) (only 0 or 1)

• Header Information:

- [Ontology](#)
- [imports](#)

• Class Intersection:

- [intersectionOf](#)

• Versioning:

- [versionInfo](#)
- [priorVersion](#)
- [backwardCompatibleWith](#)
- [incompatibleWith](#)
- [DeprecatedClass](#)
- [DeprecatedProperty](#)

• Annotation Properties:

- [rdfs:label](#)
- [rdfs:comment](#)
- [rdfs:seeAlso](#)
- [rdfs:isDefinedBy](#)
- [AnnotationProperty](#)
- [OntologyProperty](#)

• Datatypes

- [xsd datatypes](#)

OWL DL and Full

Class Axioms:

-
- [oneOf, dataRange](#)
 - [disjointWith](#)
 - [equivalentClass](#)
(applied to class expressions)
 - [rdfs:subClassOf](#)
(applied to class expressions)

Boolean Combinations of Class Expressions:

- [unionOf](#)
- [complementOf](#)
- [intersectionOf](#)

Arbitrary Cardinality:

- [minCardinality](#)
- [maxCardinality](#)
- [cardinality](#)

Filler Information:

- [hasValue](#)

Resources

W3C Documents

- Guide
<http://www.w3.org/TR/owl-guide/>
- Reference
<http://www.w3.org/TR/owl-ref/>
- Semantics and Abstract Syntax
<http://www.w3.org/TR/owl-semantics/>

OWL Tutorials

- Ian Horrocks, Sean Bechhofer:
<http://www.cs.man.ac.uk/~horrocks/Slides/Innsbruck-tutorial/>
- Talis: <http://research.talis.com/2005/rdf-intro>
- Roger L. Costello, David B. Jacobs: <http://www.xfront.com/owl/>

Example Ontologies, e.g. here:

<http://www.daml.org/ontologies/>

W3C OWL References

OWL Semantics and Abstract Syntax]

- [OWL Web Ontology Language Semantics and Abstract Syntax](http://www.w3.org/TR/2004/REC-owl-semantics-20040210/), Peter F. Patel-Schneider, Patrick Hayes, and Ian Horrocks, Editors. W3C Recommendation, 10 February 2004, <http://www.w3.org/TR/2004/REC-owl-semantics-20040210/>.
[Latest version](http://www.w3.org/TR/2004/REC-owl-semantics-20040210/) available at <http://www.w3.org/TR/owl-semantics/>.

[OWL Overview]

- [OWL Web Ontology Language Overview](http://www.w3.org/TR/2004/REC-owl-features-20040210/), Deborah L. McGuinness and Frank van Harmelen, Editors. W3C Recommendation, 10 February 2004, <http://www.w3.org/TR/2004/REC-owl-features-20040210/>.
[Latest version](http://www.w3.org/TR/2004/REC-owl-features-20040210/) available at <http://www.w3.org/TR/owl-features/>.

[OWL Reference]

- [OWL Web Ontology Language Reference](http://www.w3.org/TR/2004/REC-owl-ref-20040210/), Mike Dean and Guus Schreiber, Editors. W3C Recommendation, 10 February 2004, <http://www.w3.org/TR/2004/REC-owl-ref-20040210/>.
[Latest version](http://www.w3.org/TR/2004/REC-owl-ref-20040210/) available at <http://www.w3.org/TR/owl-ref/>.

[OWL Requirements]

- [OWL Web Ontology Language Use Cases and Requirements](http://www.w3.org/TR/2004/REC-webont-req-20040210/), Jeff Heflin, Editor. W3C Recommendation, 10 February 2004, <http://www.w3.org/TR/2004/REC-webont-req-20040210/>.
[Latest version](http://www.w3.org/TR/2004/REC-webont-req-20040210/) available at <http://www.w3.org/TR/webont-req/>.

[OWL Test Cases]

- [OWL Web Ontology Language Test Cases](http://www.w3.org/TR/2004/REC-owl-test-20040210/), Jeremy J. Carroll and Jos De Roo, Editors. W3C Recommendation, 10 February 2004, <http://www.w3.org/TR/2004/REC-owl-test-20040210/>.
[Latest version](http://www.w3.org/TR/2004/REC-owl-test-20040210/) available at <http://www.w3.org/TR/owl-test/>.